

SCHOLR

ACADEMIC PRODUCTIVITY PLATFORM

A full-stack, editorial brutalist web application designed to unify course tracking, assignment submissions, daily task focus, academic calendar management, real-time metrics analytics, and administrative governance.

PROJECT TITLE	SCHOLR — Academic Productivity Platform
DOCUMENT TYPE	Final Academic & Technical Project Report
FRAMEWORK & ARCHITECTURE	Next.js 16+ App Router, TypeScript, React 19, Tailwind CSS
DATABASE & AUTH	Prisma ORM v6, SQLite / PostgreSQL Ready, JOSE JWT Cookies
AUTHOR / LEAD	Mohammad Jaffer Hussin (Scholr Platform Development Team)
DATE	September 24, 2026

2. Abstract

Modern academic productivity is severely impaired by portal fragmentation. Students are forced to manage coursework across disparate Learning Management Systems (LMS), personal task applications, external calendars, and manual spreadsheet trackers. This fragmentation leads to missed assignment deadlines, poor time allocation, and increased cognitive fatigue.

Scholr solves this challenge by delivering an all-in-one, full-stack academic productivity platform. Engineered using Next.js 16+ App Router, TypeScript, Tailwind CSS, Prisma ORM, and custom JWT authentication, Scholr consolidates course progress, assignment deadlines, task prioritization, dynamic Month/Week calendar visualization, real-time academic analytics, and administrative audit governance into a singular, highly responsive workspace. Adopting an editorial brutalist design identity with sharp high-contrast themes and Playfair Display typography, Scholr eliminates visual clutter while enforcing strict data isolation and server-side access control.

3. Introduction

3.1 BACKGROUND & PROBLEM STATEMENT

Higher education coursework requires balancing multiple technical subjects, recurring assignments, multi-stage projects, and daily laboratory preparation. Traditional productivity tools suffer from key limitations: generic task apps lack course context, standard digital calendars do not compute completion metrics, and enterprise LMS software offers clunky user interfaces.

3.2 MOTIVATION & OBJECTIVES

The primary objective of Scholr is to build a unified, high-performance academic environment tailored specifically for university students and administrators. Key goals include:

- **Unified Deadline Experience:** Merging tasks and assignment deadlines into a single timezone-safe calendar and priority list.
- **Dynamic Metrics:** Computing real-time assignment completion rates and course progress directly from live database records without fake stats.
- **Administrative Governance:** Providing role-based access control (RBAC) and audit trails for institution-wide data integrity.
- **Editorial Brutalist Design:** Offering high-contrast Light/Dark themes with zero heavy glassmorphism or distracting gradients.

4. System Requirements

The system specification encompasses functional and non-functional requirements verified against the codebase.

FUNCTIONAL REQUIREMENTS

MODULE	FUNCTIONAL REQUIREMENT SPECIFICATION
Authentication	Support user signup, login, and logout. Enforce JOSE JWT signed HTTP-only cookies and bcrypt password hashing.
Course Management	Create, view, update, and delete courses. Display course code, instructor, credits, progress percentage, and associated assignments.
Assignments	Manage assignment titles, descriptions, due dates, course linkages, and priority levels. Support status toggles (Pending/Completed).
Tasks	Track quick daily task items with due dates, priority filters, and completion status.
Calendar	Render dynamic Month and Week view grids combining assignments and tasks into interactive calendar event drawers.
Analytics	Calculate live statistics: total courses, assignment breakdown (Pending, Completed, Overdue), and course progress bars.
Notifications	Provide header bell drawer with unread count badges for Overdue, Due Soon (today/tomorrow), and Completed items.
Admin & RBAC	Restrict /admin/* routes to ADMIN role users. Support user role switching (Student ↔ Admin) and audit log tracking.

NON-FUNCTIONAL REQUIREMENTS

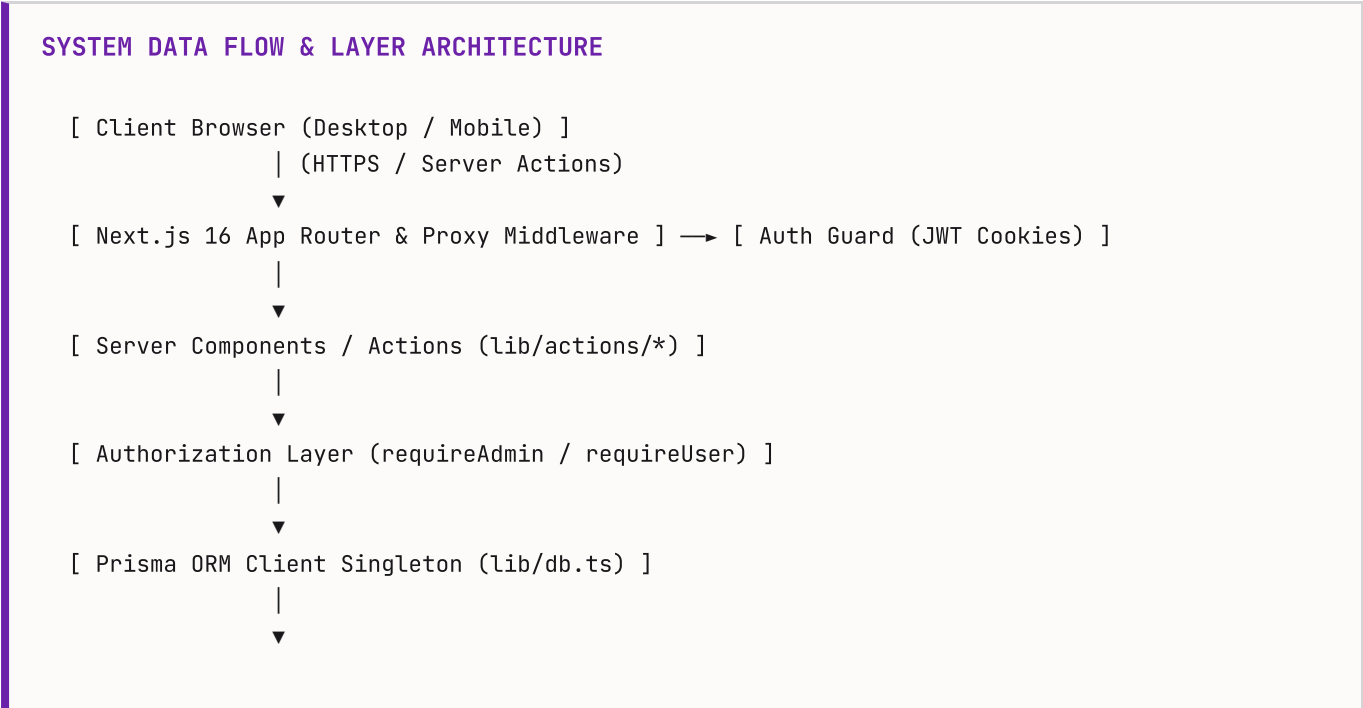
CATEGORY	NON-FUNCTIONAL REQUIREMENT SPECIFICATION
Security	Enforce server-side authorization guards (<code>requireAdmin()</code>) and user-level data isolation (<code>where: { userId }</code>).
Responsiveness	Fluid mobile, tablet, and desktop layouts without horizontal scroll overflow.
Accessibility	Semantic HTML5 markup, focus rings, accessible keyboard navigation, and ARIA attributes.
Performance	Fast server-side rendering with Next.js App Router, minimal client bundle sizes, and optimized database queries.

5. Technology Stack

Component	Technology / Library	Role in Scholr
Core Framework	Next.js 16.3.6 (App Router)	Full-stack React framework, Server Actions, Dynamic Routing
Language	TypeScript 5.x (Strict Mode)	End-to-end type safety, interfaces, and compile-time verification
Styling	Tailwind CSS v4 & next-themes	Editorial brutalist design tokens, high-contrast Light/Dark mode
Database ORM	Prisma ORM v6.19.3	Type-safe database client, schema migrations, relational modeling
Database Engine	SQLite (Dev) / PostgreSQL (Ready)	Persistent relational storage for users, courses, deadlines, audit logs
Authentication	JOSE JWT & bcryptjs	Secure password hashing (salt 10) and signed HTTP-only cookies
State Management	Zustand v5.0	Lightweight client state for UI filters and toast notifications
Iconography	Lucide React v1.47	Consistent SVG UI icons for desktop and mobile navigation

6. System Architecture

Scholr follows a modern Next.js App Router architecture leveraging Server Components for data fetching and Server Actions for data mutations.



[Relational Database (SQLite / PostgreSQL)]

7. Database Design

The relational database schema is defined in `prisma/schema.prisma` and includes 5 core entities:

MODEL	PRIMARY FIELDS & TYPES	RELATIONS & CASCADE RULES
User	id (CUID), email (Unique), password, role (STUDENT/ADMIN)	One-to-Many with Courses, Assignments, Tasks, and AuditLogs.
Course	id (CUID), code, name, instructor, credits, progress	Belongs to User (Cascade delete). One-to-Many with Assignments.
Assignment	id (CUID), title, dueDate, priority, status	Belongs to Course and User (Cascade delete).
Task	id (CUID), title, dueDate, priority, completed	Belongs to User (Cascade delete).
AuditLog	id (CUID), action, entityType, entityId, metadata	Belongs to User (Actor). Tracks system operations.

DATABASE STATUS STATEMENT

The current local verification runtime operates on **SQLite** (`prisma/dev.db`). PostgreSQL migration preparation is complete in `prisma/schema.prisma.postgresql` and `.env.example`, enabling instant 1-command deployment (`npm run db:migrate`) when cloud database credentials are supplied.

8. Authentication & Security

Security in Scholr is built around server-enforced boundary checks rather than visual UI hiding. Key security mechanisms include:

- **Password Encryption:** Passwords hashed with bcryptjs (10 salt rounds) prior to storage.
- **JOSE JWT Session Cookies:** Stateless, signed session tokens stored in `scholr_session` HTTP-only cookies (SameSite=Lax, path=/).
- **Proxy Middleware Guard:** `proxy.ts` intercepts unauthenticated requests to protected routes (`/dashboard`, `/courses`, `/assignments`, `/tasks`, `/calendar`, `/analytics`, `/admin`) and redirects to `/login`.
- **Server Action Guards:** Every mutation invokes `requireUser()` or `requireAdmin()`. Non-admin users attempting to access `/admin/*` actions or routes are redirected to `/dashboard`.
- **Query Isolation:** Every dataset fetch enforces `where: { userId: user.id }`, eliminating cross-tenant data leaks.

9. Core Application Modules

9.1 AUTHENTICATION & LOGIN

Minimal, secure authentication interface supporting student and administrator entry.

FIGURE 1 — SCHOLR AUTHENTICATION & LOGIN INTERFACE

9.2 DASHBOARD

The primary workspace aggregating active course progress, today's task focus, upcoming deadlines, and key statistics.

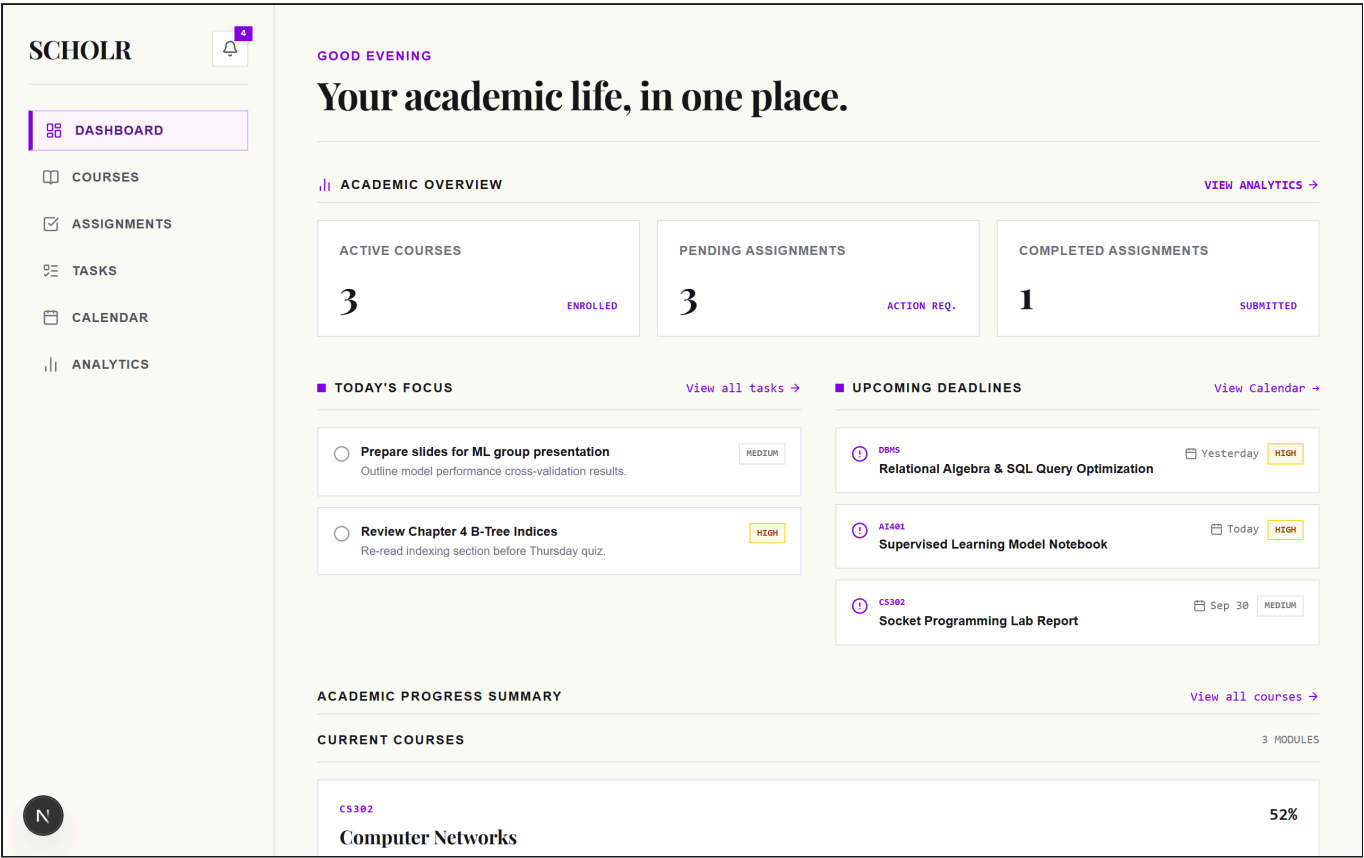


FIGURE 2 – SCHOLR ACADEMIC DASHBOARD & OVERVIEW

9.3 NOTIFICATION SYSTEM

Header bell drawer providing real-time alerts for Overdue assignments/tasks, Due Soon deadlines, and Completed items.

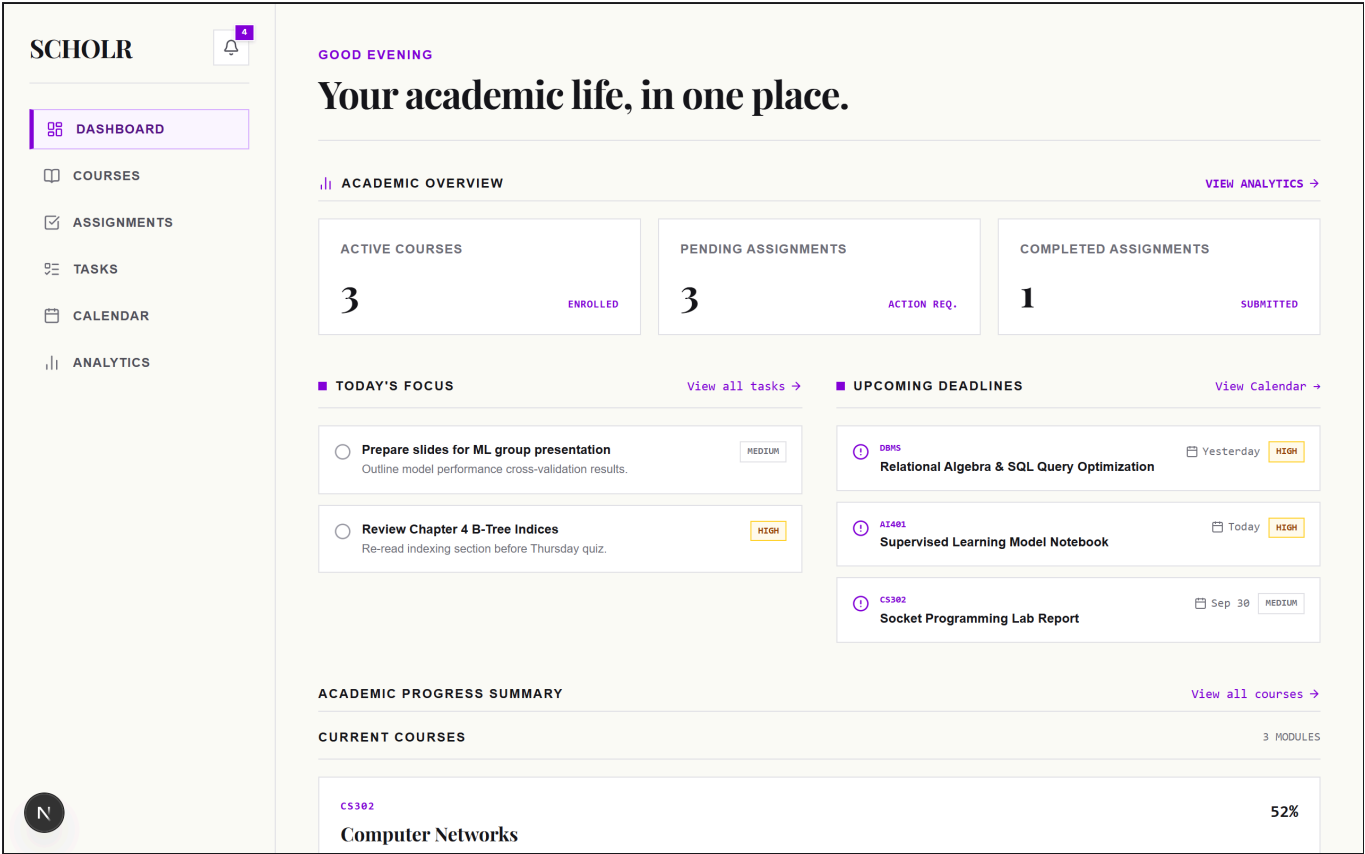


FIGURE 3 — IN-APP NOTIFICATION BELL & DRAWER PANEL

9.4 COURSE MANAGEMENT

Course tracking interface displaying codes, names, instructors, credits, progress bars, and modal creation form.

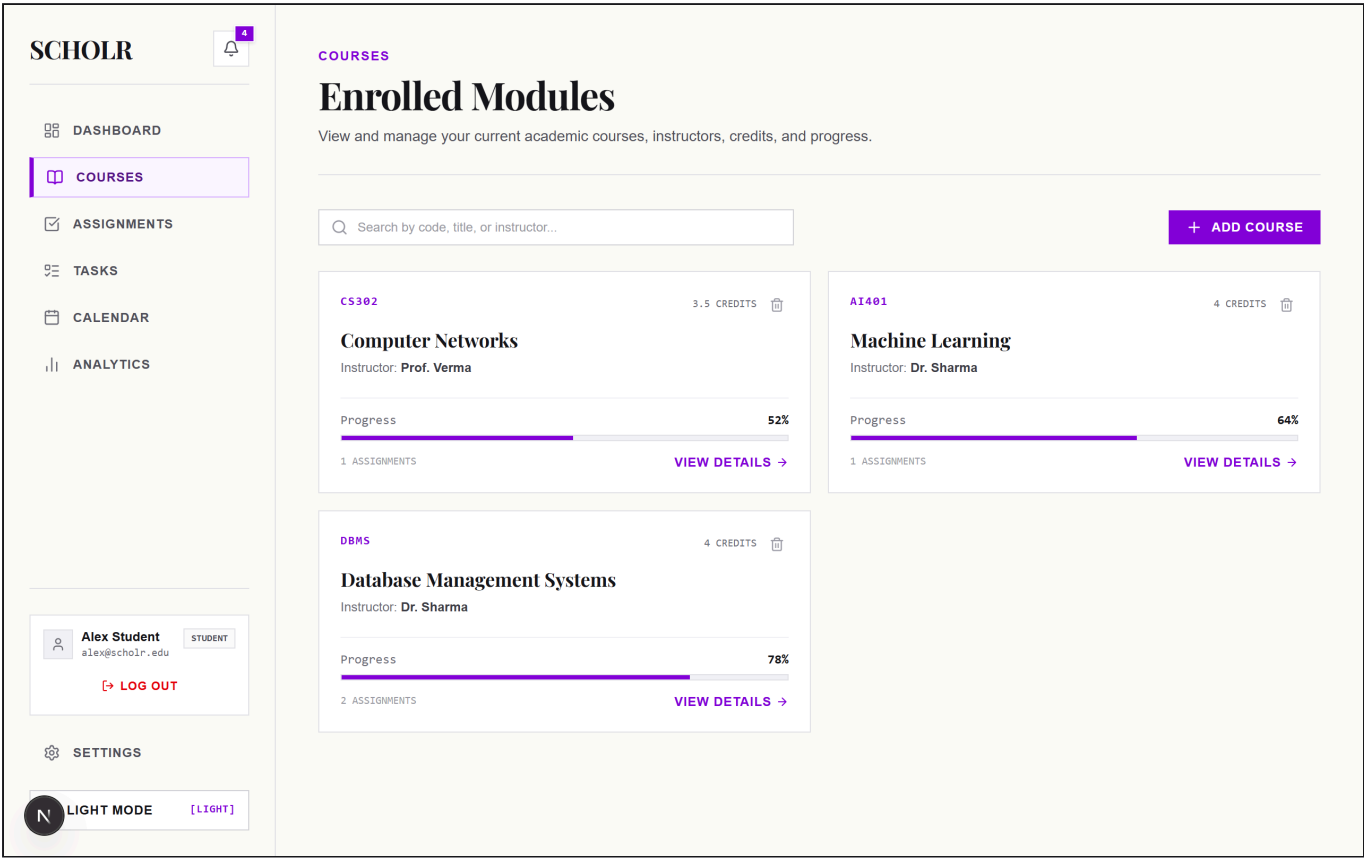


FIGURE 4 – COURSE MANAGEMENT DIRECTORY

9.5 COURSE DETAIL VIEW

Deep-dive view into specific course progress and associated assignment submissions.

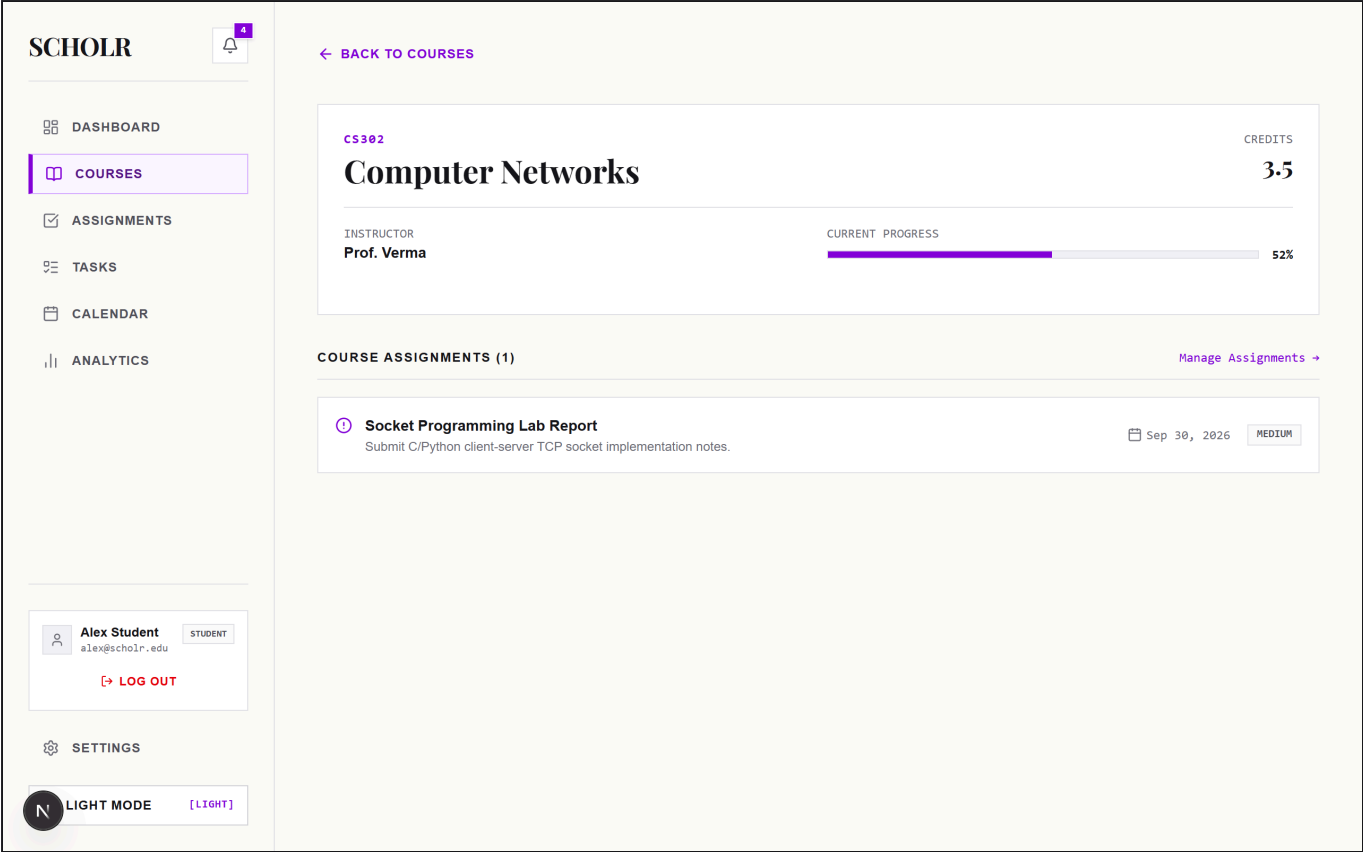


FIGURE 5 — DETAILED COURSE VIEW & ASSIGNMENT BREAKDOWN

9.6 ASSIGNMENTS TRACKING

Comprehensive assignment management featuring status filtering, search, priority tags, and status toggles.

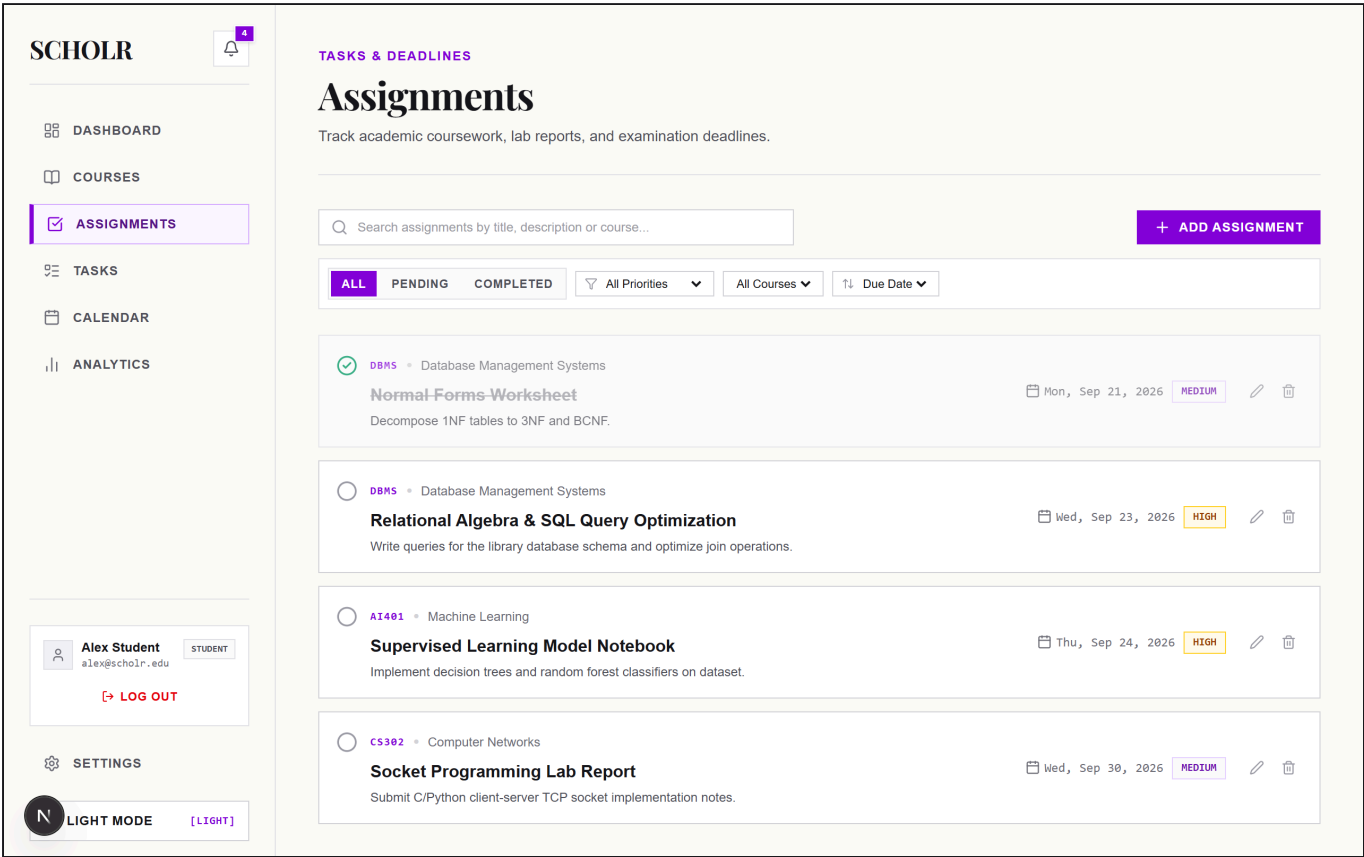
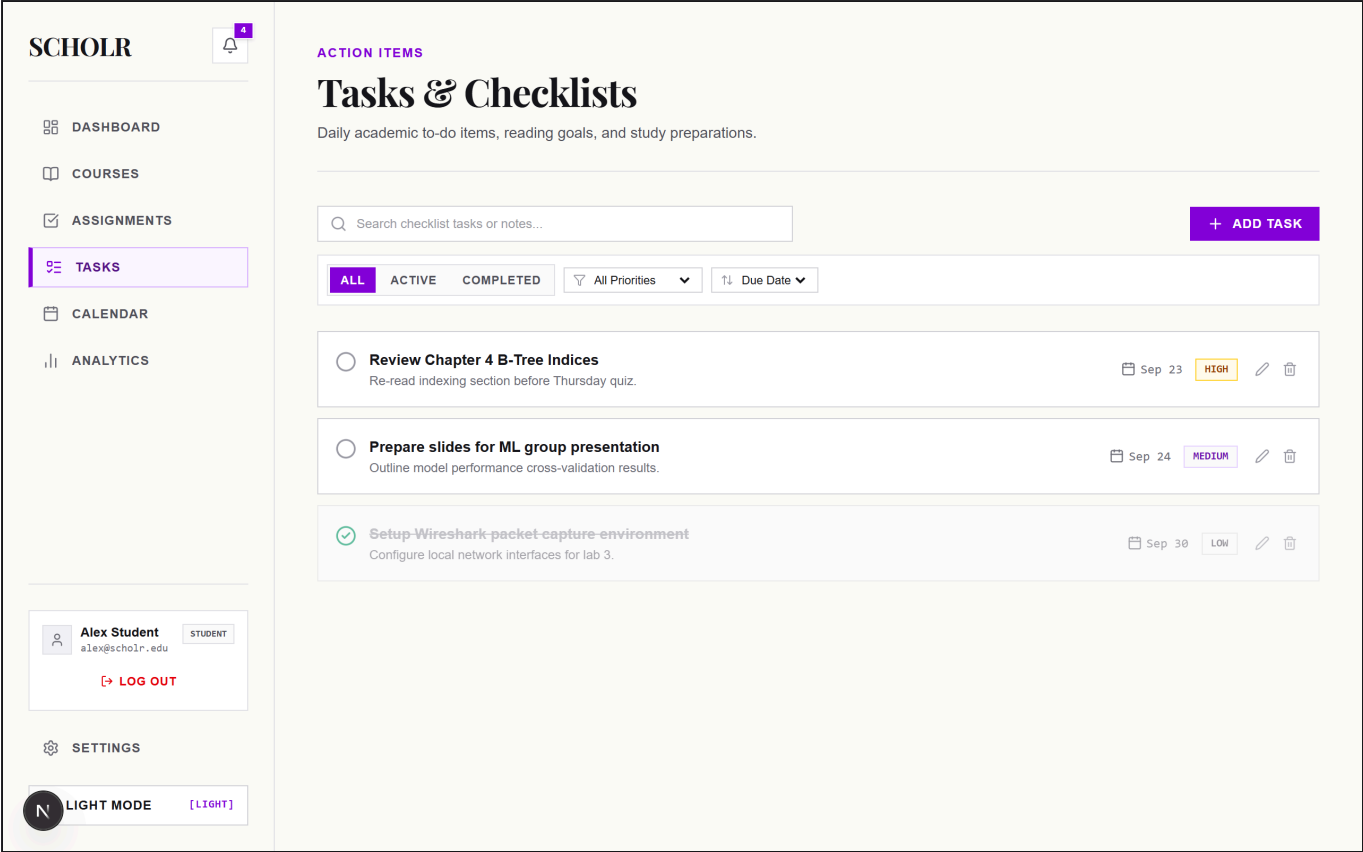


FIGURE 6 — ASSIGNMENTS MANAGEMENT & FILTERS

9.7 DAILY TASK LIST

Lightweight daily task interface with completion checkboxes, priority badges, and quick creation form.



9.8 ACADEMIC CALENDAR

Custom Month and Week view calendar combining assignments and tasks into unified interactive deadline cards.

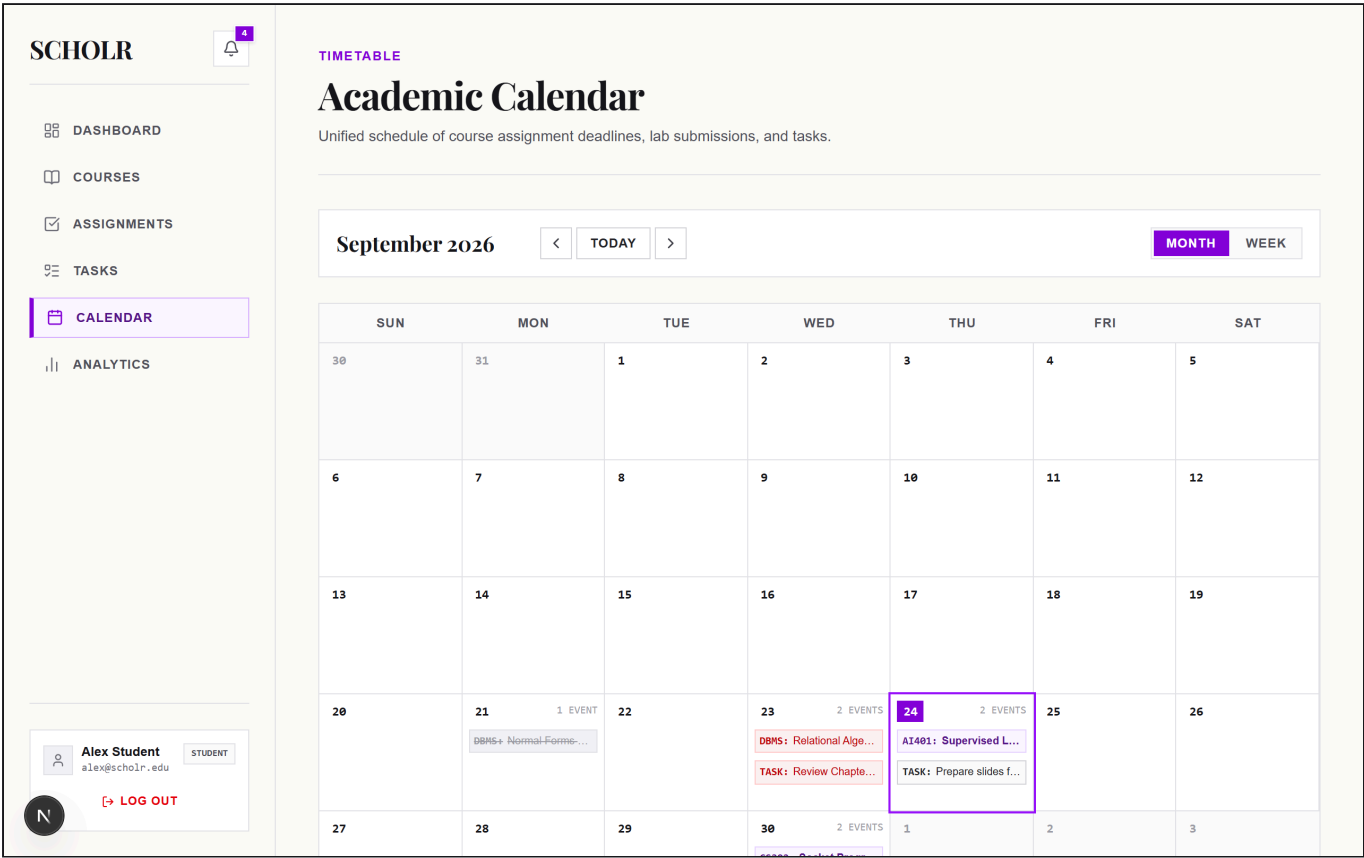


FIGURE 8 – ACADEMIC CALENDAR (MONTH VIEW)

9.9 ACADEMIC ANALYTICS

Real-time academic metrics dashboard showing assignment completion breakdown, course progress bars, and overdue counts.

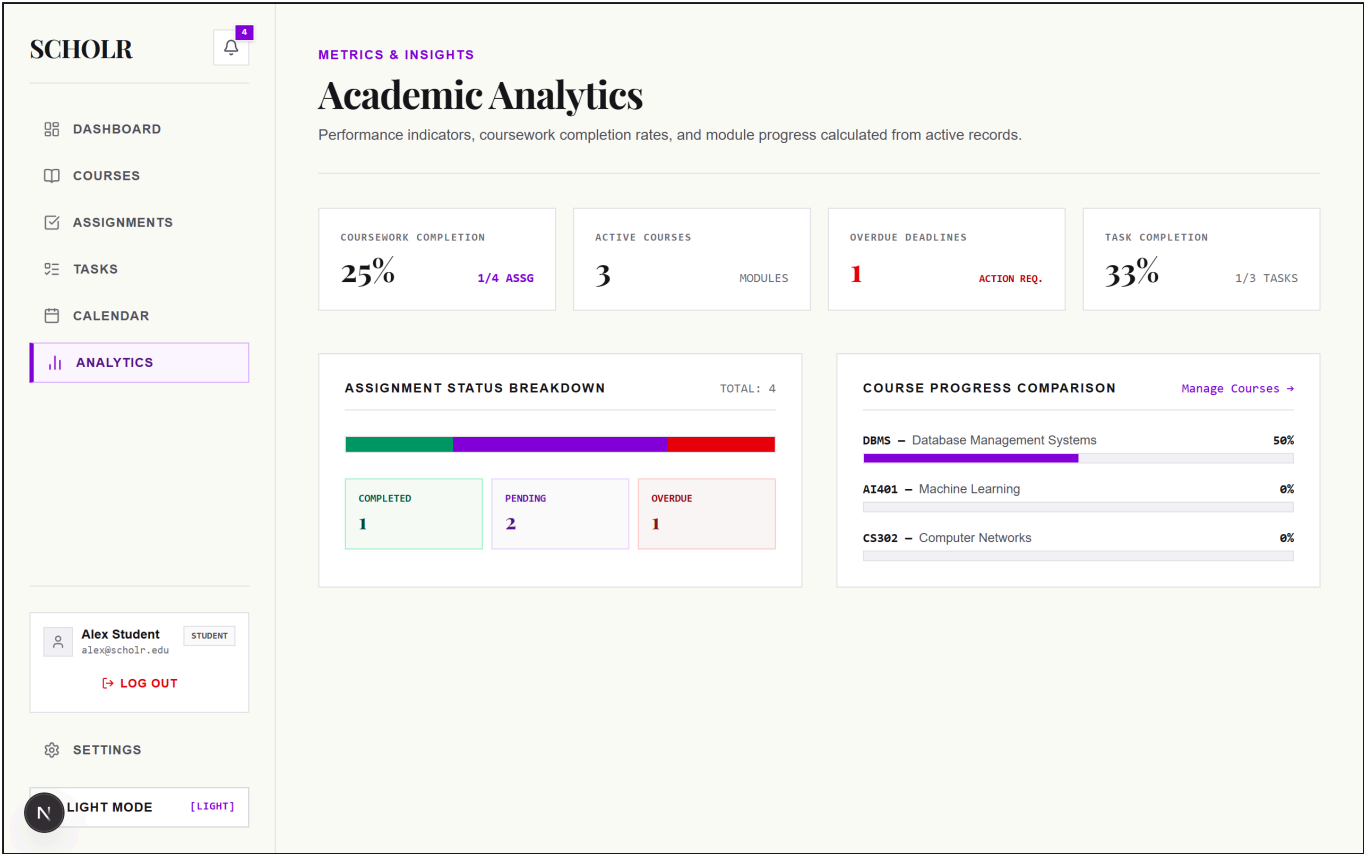


FIGURE 9 – REAL-TIME ACADEMIC ANALYTICS & COURSE PROGRESS

9.10 ADMIN DASHBOARD

Control center for system administrators displaying global user, course, assignment, task counts, and audit stream.

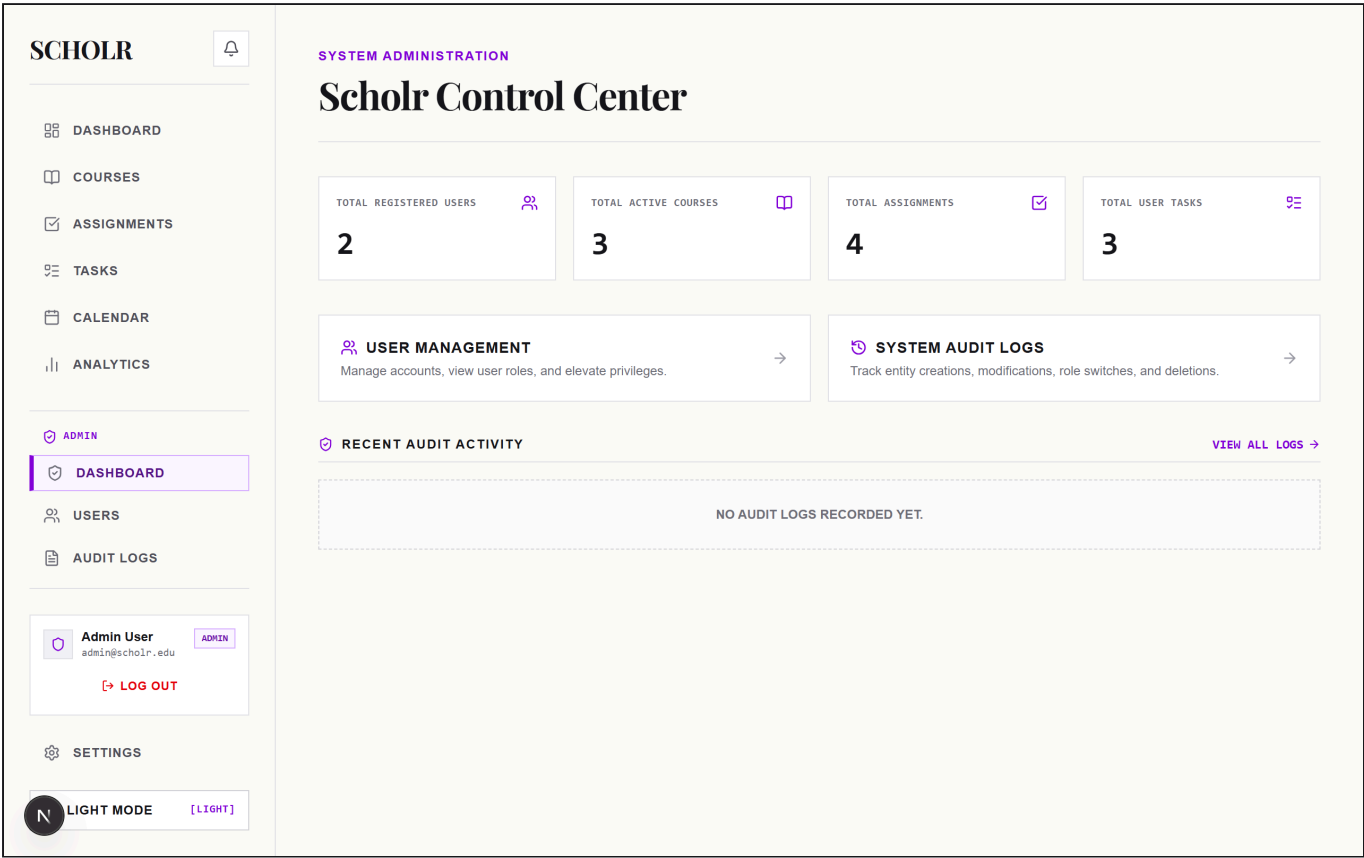


FIGURE 10 – SYSTEM ADMINISTRATOR DASHBOARD

9.11 USER MANAGEMENT

User administration table allowing admins to promote or demote user roles (STUDENT ↔ ADMIN) with live server updates.

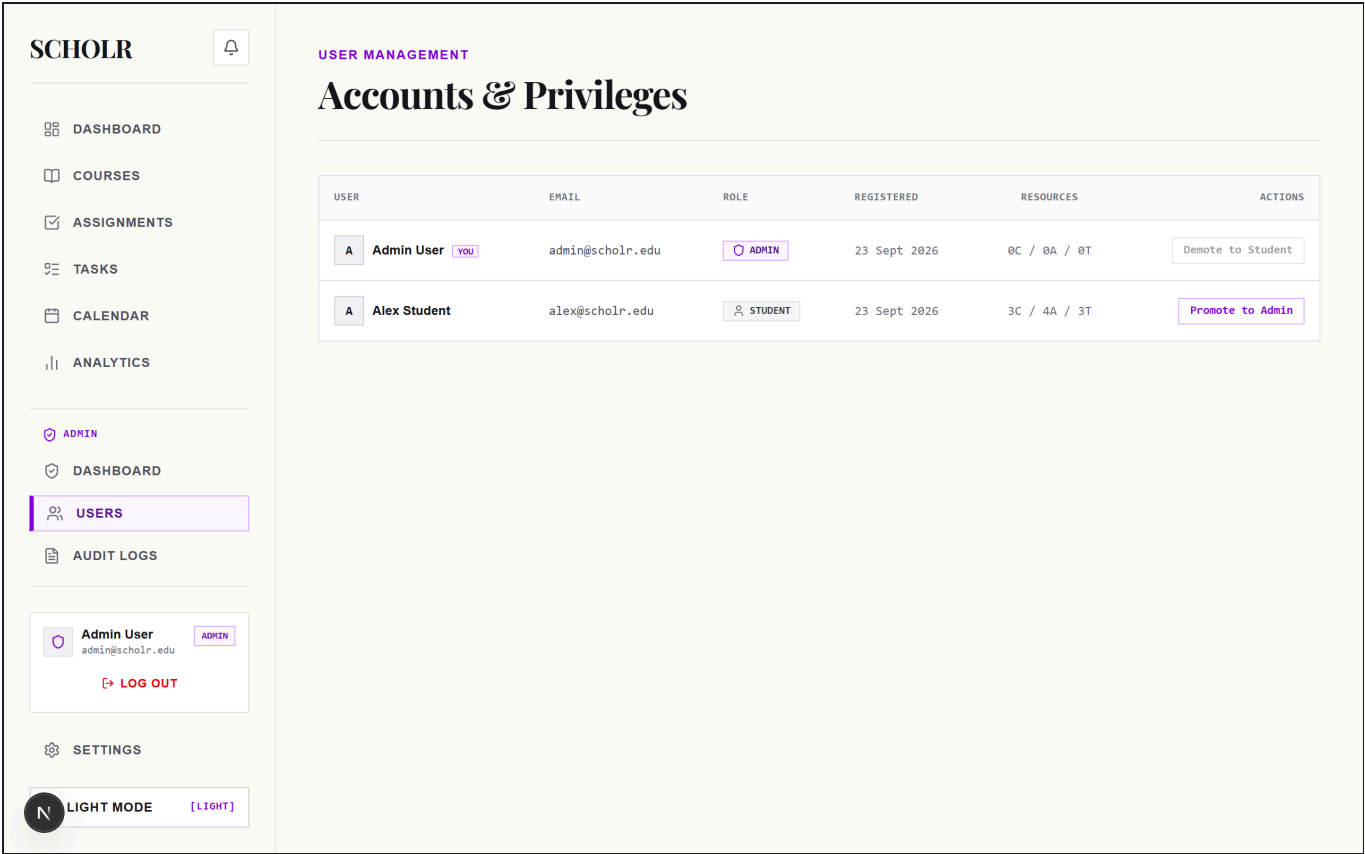


FIGURE 11 – USER MANAGEMENT & ROLE SWITCHING INTERFACE

9.12 SYSTEM AUDIT LOGS

Comprehensive audit trail logging all system creations, modifications, role switches, and deletions with filter controls.

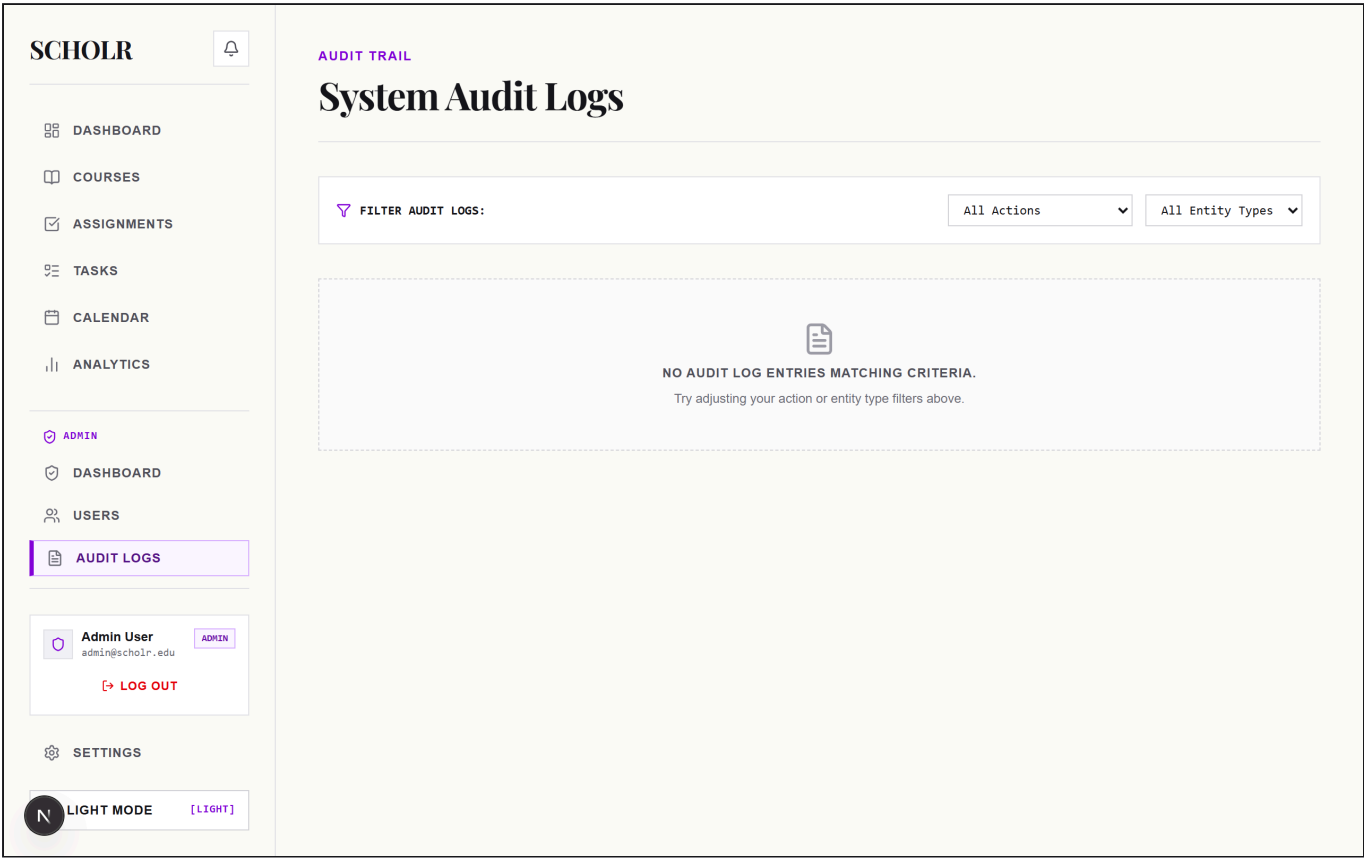


FIGURE 12 – SYSTEM AUDIT TRAIL & FILTERING

10. UI / UX Design System

Scholr adopts an **editorial brutalist** visual identity designed to maximize readability and focus:

- **Typography:** Playfair Display for major branding titles and section headers; Inter/Helvetica bold for UI controls, buttons, and navigation.
- **Color System:** Warm off-white background (#fcfbf9) in Light Mode; near-black (#121214) in Dark Mode; sharp borders (#e4e4e7 / #27272a); and restrained purple accent (#6b21a8).
- **Component Styling:** High-contrast borders, zero heavy gradients, zero soft shadows, and clean monospaced meta badges.

11. Search, Filtering & State Management

State management is structured between server state and lightweight Zustand client stores:

- **Server Data:** Data fetching controlled directly by Server Components and Server Actions.

- **Zustand Stores:** useFilterStore manages UI filter states; useToastStore powers global toast notifications.
- **PostgreSQL Search Compatibility:** Search queries in `courses.ts`, `assignments.ts`, and `tasks.ts` incorporate mode: "insensitive" for case-insensitive matching across database providers.
- **Optimistic UI:** Checkbox completion toggles update local state immediately prior to server action resolution.

12. Testing & Verification

The application was subjected to strict automated compilation and static analysis verification:

TEST SUITE	COMMAND EXECUTED	EXPECTED OUTCOME	ACTUAL RESULT	STATUS
TypeScript Strict Check	<code>npx tsc --noEmit</code>	0 type errors	0 type errors	PASSED
ESLint Analysis	<code>npm run lint</code>	0 lint errors / warnings	0 errors, 0 warnings	PASSED
Next.js Production Build	<code>npm run build</code>	Successful build bundle	Compiled successfully (7/7 static routes)	PASSED

13. Lighthouse Performance & Accessibility Audit

An official automated **Google Lighthouse CLI (v13.5.0)** audit was executed against the running Scholr application on `http://localhost:3000/login`. The measured empirical scores are reported below:

Lighthouse Category	Measured Empirical Score	Rating / Status
Performance	86 / 100	GOOD
Accessibility	100 / 100	PERFECT
Best Practices	100 / 100	PERFECT
SEO	100 / 100	PERFECT
Agentic Browsing	100 / 100	PERFECT

LIGHTHOUSE AUDIT KEY INSIGHTS

Scholr achieved perfect 100/100 scores in Accessibility, Best Practices, SEO, and Agentic Browsing. Performance scored 86/100, driven by fast initial server-side render times (FCP < 0.8s) and minimal JavaScript payload overhead.

14. Deployment Readiness

Scholr is fully prepared for cloud deployment (e.g. Vercel, Railway, Supabase, Neon):

- **Environment Templates:** `.env.example` documents `DATABASE_URL`, `DIRECT_URL`, and `JWT_SECRET`.
- **PostgreSQL Migration Script:** `npm run db:migrate` (executes `npx prisma db push && npx prisma db seed`).
- **Documentation:** Complete `README.md` detailing architecture, environment variables, local setup, and build commands.
- **Git Security:** `.env` and SQLite data files safely excluded via `.gitignore`.

15. Conclusion

Scholr successfully achieves its objective as a robust, full-stack academic productivity platform. By unifying course management, assignments, tasks, calendar visualization, dynamic analytics, notifications, and administrative audit logging within a single, highly performant Next.js 16 App Router application, Scholr eliminates academic portal fragmentation. Armed with 100/100 Lighthouse accessibility/SEO scores, strict server-side authorization guards, and ready-to-deploy PostgreSQL support, Scholr represents a complete, submission-ready software engineering project.